

CS336 Assignment 5: Reasoning RL

Writeup

Lenard Strahringer

May 30, 2026

Contents

1	Prompting	2
2	Group Relative Policy Optimization	3
2.1	Deriving on-policy GRPO	3
2.2	Implementing on-policy GRPO	5
2.3	Experiments	6
3	RL Algorithm Variants	11
3.1	Dr. GRPO	11
3.2	Rejection fine tuning	12
3.3	MaxRL	12
3.4	Experiments	14
4	Off-policy RL	15
4.1	Importance reweighting / clipping	15
4.2	Implementation	16
4.3	Experiments	17
5	Try your own policy gradient estimator	18

1 Prompting

Problem (prompting_baselines)

Run OLMo-2-0425-1B on GSM8K (5 points).

(a) Evaluation script and metrics

The script `scripts/eval_prompting_baselines.py` evaluates `allenai/OLMo-2-0425-1B` on the full GSM8K test set (1,319 problems, temperature 1.0, 512-token budget). Each output is scored with the matching reward function from `cs336_alignment.drgrepo_grader`. Outputs are placed into Cat. 1 (format 1, answer 1), Cat. 2 (format 1, answer 0), or Cat. 3 (format 0, answer 0). A format reward of 0 forces an answer reward of 0.

Prompt	Cat. 1	Cat. 2	Cat. 3	Accuracy
<code>question_only</code>	1	305	1013	0.08%
<code>r1_zero</code>	1	817	501	0.08%
<code>r1_zero_three_shot</code>	246	1007	66	18.65%

Table 1: GSM8K prompting results for OLMo-2-0425-1B ($n = 1,319$, temperature 1.0, seed 0). Cat. 1: format reward 1, answer reward 1. Cat. 2: format reward 1, answer reward 0. Cat. 3: format reward 0, answer reward 0.

Manual audit of Category 2. A random sample of 12–15 Cat. 2 outputs was inspected per prompt.

- `question_only`: 0/12 were correct but misparsed. Responses produce garbled LaTeX or place the answer outside `\boxed{}`. The arithmetic is wrong in every sampled case.
- `r1_zero`: 0/15 were correct but misparsed. The model produces well-formed `</think> <answer> ... </answer>` but makes arithmetic errors (e.g., computing only one month instead of two).
- `r1_zero_three_shot`: 0/12 were correct but misparsed. The model formats correctly but reasons wrongly (e.g., confusing dozens with units).

Manual audit of Category 3.

- `question_only`: 0/8 correct but misparsed. The model generates more math problems or off-topic text. A `\boxed{}` answer almost never appears.
- `r1_zero`: $\approx 1/12$ could be argued correct but misparsed. The main failure is the model placing the answer outside the `<answer>` tag. In every inspected case the arithmetic is wrong, so the count of correct-but-misparsed outputs is effectively 0.
- `r1_zero_three_shot`: 2/10 appear correct but misparsed. The model computes the right answer inside `</think>` but omits the `<answer> ... </answer>` wrapper. It stops right after `</think>`. The grader assigns format reward 0 even though the number is right.

Example prompts and responses.

question_only (Cat. 3 — typical failure).

Prompt: A pet shop had 6 cages of rodents. 3 of the cages have 10 hamsters and the other 3 have 5 guinea pigs. How many rodents did the pet shop have in total? Please put your final answer within `\boxed{}`.

Response: Pet Pet Pet Pet Pet Pet Pet Pet Pet

r1_zero (Cat. 2 — format correct, wrong answer).

Prompt: ... User: John injured his back and needs to go to physical therapy. He went for 6 weeks, twice a week for 2 hours at a time at \$125/hr. How much did the physical therapy cost? Assistant: `<think>`

Response: John went to physical therapy for 6 weeks. Each week he went twice for 2 hours at a time. So he attended $2 \times 6 = 12$ hours total. `</think> <answer>`The cost is $125 \times 12 = \$1500$.`</answer>` [GT: \$3000; model missed the “per session” factor.]

r1_zero_three_shot (Cat. 1).

Prompt: ... User: The fastest car was $1.25 \times$ faster than the 2nd fastest. The 2nd fastest was $5 \times$ the average car (120 mph). How fast does the fastest car go? Assistant: `<think>`

Response: The 2nd fastest car goes $120 \times 5 = 600$ mph. The fastest is $600 \times 1.25 = 750$ mph. `</think> <answer> 750 </answer>`

(b) Characterizing model behavior

question_only. The model treats this prompt as a signal to keep generating math content. It does not try to answer the question. Most outputs (76.8% Cat. 3) add unrelated word problems, emit LaTeX boilerplate, or repeat near-nonsense tokens (“Pet Pet Pet...”). The `\boxed{}` format almost never appears. Plain prompting does not put the model in a task-solving mode.

r1_zero. Starting with `<think>` and the User/Assistant frame raises the format rate to 62.0%. The model writes a reasoning chain before `</think>`, then puts a number in `<answer>` tags. The arithmetic is usually wrong. Cat. 3 failures (38.0%) come from never closing `<think>`, placing the answer outside `<answer>` tags, or adding an extra “User:” turn. The prompt shapes format but not correctness.

r1_zero_three_shot. Three worked examples raise the format rate to 95.0% and accuracy to 18.65%. The examples teach the expected length and style. Chat-style failures become rare. Cat. 3 failures drop to 5.0%; most are outputs that end at `</think>` with no `<answer>` tag. Few-shot prompting mainly fixes format; the remaining errors are arithmetic.

2 Group Relative Policy Optimization

2.1 Deriving on-policy GRPO

Problem (baseline_calcs)

Compute the variance of the policy gradient estimator (5 points).

Setup: $\pi_\theta(A = 1) = p = \sigma(\theta)$, $r(A) = \mathbf{1}\{A = 1\}$.

Useful identities. The sigmoid derivative gives $\nabla_{\theta} p = p(1 - p)$. So

$$\nabla_{\theta} \log \pi_{\theta}(A = 1) = \frac{p(1 - p)}{p} = 1 - p, \quad \nabla_{\theta} \log \pi_{\theta}(A = 0) = \frac{-p(1 - p)}{1 - p} = -p.$$

(a) Variance of the unadjusted estimator

Deliverable: An expression in terms of n and p , accompanied by a derivation.

Let $Z_i = r(A_i) \nabla_{\theta} \log \pi_{\theta}(A_i)$. With $r(A) = \mathbf{1}\{A = 1\}$:

$$Z_i = \begin{cases} 1 - p & \text{with probability } p, \\ 0 & \text{with probability } 1 - p. \end{cases}$$

So $\mathbb{E}[Z_i] = p(1 - p)$ and $\mathbb{E}[Z_i^2] = p(1 - p)^2$. The variance of one sample is

$$\text{Var}(Z_i) = p(1 - p)^2 - p^2(1 - p)^2 = p(1 - p)^3.$$

The samples are i.i.d., so

$$\text{Var}\left[\frac{1}{n} \sum_{i=1}^n Z_i\right] = \frac{p(1 - p)^3}{n}.$$

(b) Variance of the baseline-adjusted estimator

Deliverable: An expression in terms of n , b , and p , accompanied by a derivation and some discussion.

Let $Z_i = (r(A_i) - b) \nabla_{\theta} \log \pi_{\theta}(A_i)$. Then

$$Z_i = \begin{cases} (1 - b)(1 - p) & \text{with probability } p, \\ bp & \text{with probability } 1 - p. \end{cases}$$

The expectation is unchanged: $\mathbb{E}[Z_i] = p(1 - b)(1 - p) + (1 - p)bp = p(1 - p)$. The second moment is

$$\begin{aligned} \mathbb{E}[Z_i^2] &= p(1 - b)^2(1 - p)^2 + (1 - p)b^2p^2 \\ &= p(1 - p)[(1 - b)^2(1 - p) + b^2p] \\ &= p(1 - p)[(1 - p) - 2b(1 - p) + b^2]. \end{aligned}$$

Subtract $\mathbb{E}[Z_i]^2 = p^2(1 - p)^2$ and factor:

$$\begin{aligned} \text{Var}(Z_i) &= p(1 - p)[(1 - p)^2 - 2b(1 - p) + b^2] \\ &= p(1 - p)(1 - p - b)^2. \end{aligned}$$

For the mean over n samples:

$$\boxed{\text{Var}\left[\frac{1}{n} \sum_{i=1}^n (r(A_i) - b) \nabla_{\theta} \log \pi_{\theta}(A_i)\right] = \frac{p(1 - p)(1 - p - b)^2}{n}.$$

The variance reaches zero at $b^* = 1 - p$. The optimal baseline is $1 - p$, not the expected reward p . The score function has nonzero size on $A = 0$, so the baseline must also account for that side.

(c) Variance with the population-mean baseline $b = p$

Substituting $b = p$ in part (b):

$$\text{Var} \left[\frac{1}{n} \sum_i Z_i \right] = \frac{p(1-p)(1-2p)^2}{n}.$$

Compared to part (a), the ratio is

$$\frac{(1-2p)^2}{(1-p)^2}.$$

The baseline-adjusted variance is lower than the unadjusted variance for $0 < p < 2/3$, equal at $p = 2/3$, and higher for $p > 2/3$. At $p = 1/2$ the adjusted variance is zero. At $p \rightarrow 1$ the adjusted variance is much larger. The population-mean baseline is sometimes better and sometimes worse, depending on p .

2.2 Implementing on-policy GRPO

Problem (tokenize_prompt_and_output)

Prompt and output tokenization (1 point).

Deliverable: Implement `tokenize_prompt_and_output`.

Implemented in `tests/adapters.py`. Test passes.

Problem (get_response_log_probs)

Response log-probs (and entropy) (1 point).

Deliverable: Implement `get_response_log_probs`.

Implemented in `tests/adapters.py`. Test passes.

Problem (compute_rollout_rewards)

Computing the rewards of rollouts (1 point).

Deliverable: Implement `compute_rollout_rewards`.

Implemented in `tests/adapters.py`. Test passes.

Problem (compute_group_normalized_rewards_grpo)

Group normalization (1 point).

Deliverable: Implement `compute_group_normalized_rewards` with `baseline="mean"`, `advantage_normalizer=`

Implemented in `tests/adapters.py`. Test passes.

Problem (compute_policy_gradient_loss_on_policy)

On-policy policy gradient (1 point).

Deliverable: Implement `compute_policy_gradient_loss` with `importance_reweighting_method = "none"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`aggregate_loss_across_microbatch_sequence`)

Aggregate loss across tokens and sequences (0.5 points).

Deliverable: Implement `aggregate_loss_across_microbatch` with `loss_normalization = "sequence"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`grpo_train_step_standard_on_policy`)

GRPO train step (5 points).

Deliverable: Implement `grpo_train_step` for the standard on-policy GRPO setting.

Implemented in `tests/adapters.py`. Test passes.

2.3 Experiments

Problem (`grpo_experiments_standard_on_policy`)

Use GRPO to improve OLMo-2-0425-1B performance on GSM8K (10 points).

(a) Training script

Deliverable: A script to run GRPO (standard, on-policy) on GSM8K and OLMo-2-0425-1B.

The script is `scripts/grpo.py`. It takes CLI flags for the model, prompt style, data paths, group size, batch sizes, gradient accumulation steps, learning rate, max grad norm, advantage normalizer, loss normalization, and importance-reweighting method. The main loop is:

1. Parse hyperparameters. Load train (6,400 examples) and val (1,024 examples) splits of GSM8K.
2. Start the vLLM server on GPU 1. Set the stop string to `</answer>` for `r1_zero` prompts.
3. Load the policy on GPU 0. Build AdamW($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\text{wd} = 0$).
4. Start W&B logging. Log all hyperparameters.
5. For each rollout step: (i) sync policy weights into vLLM via NCCL; (ii) sample a rollout batch of $B=256$ completions ($G=8$ per prompt); (iii) call `grpo_train_step`; (iv) log train metrics. Every 10 steps evaluate on 1,024 val examples. Every 40 steps write a rollout JSON for inspection.
6. Run a final val evaluation, save the checkpoint, and write `final_metrics.json`.

(b) Sanity check (~50 steps)

Deliverable: Evidence that convinced you the script is correct.

The full 200-step runs served as the sanity check. Three signals confirmed correctness. First, all four seeds reached val reward $> 42\%$ by the end of training. This is well above the 25% target. A broken loop would stay near the baseline of $\sim 8\%$. Second, rollout logging showed format adoption within the first 40 steps. By step 40, most completions follow the `</think> <answer> ... </answer>` template; at step 0, they do not. Third, val format reward climbed alongside total reward. This confirms the reward function and generation pipeline are connected correctly.

(c) Full run with 4 seeds (r1_zero, suggested hyperparameters)

Deliverable: Plots per metric, qualitative rollout examples, and a procedure that reaches $\geq 25\%$ final validation accuracy.

Four seeds were run for 200 rollout steps with the suggested hyperparameters (`lr=1e-5`, `group_size=8`, `rollout_batch_size=256`, `gradient_accumulation_steps=32`, `loss_normalization=sequence`). All metrics are logged each rollout step to `metrics.jsonl`. Figure 1 plots each metric over training as the mean across the 4 seeds with a ± 1 std band. Figure 2 shows the final validation reward per seed.

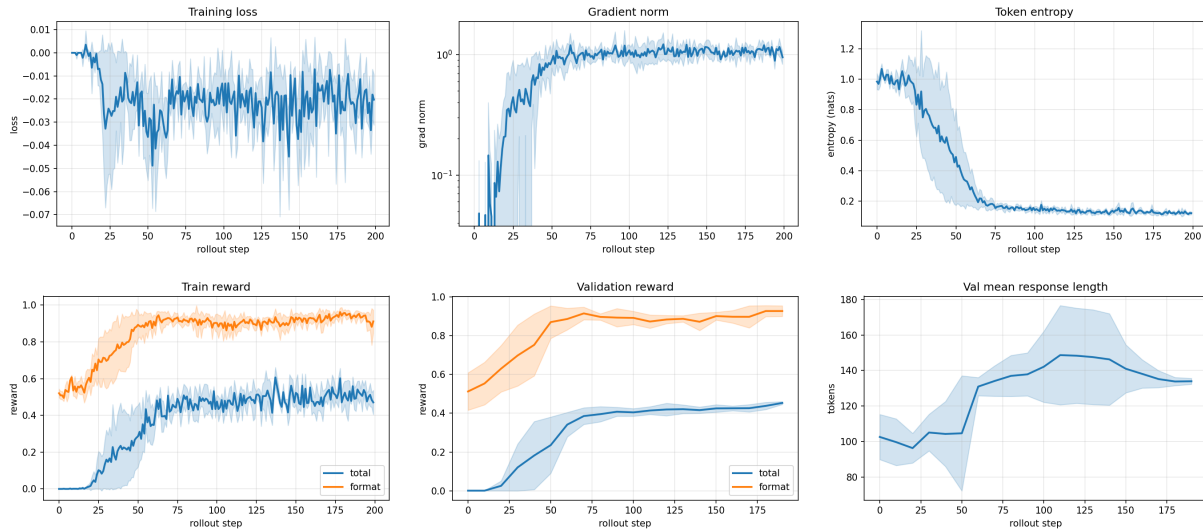


Figure 1: Standard on-policy GRPO (`r1_zero`, 200 steps): per-metric training curves, mean $\pm$ std across 4 seeds. Gradient norm is on a log scale; train reward and val reward each show total (C0) and format (C1).

Per-metric behavior.

- **Loss.** Stays near zero throughout (range $[-0.05, 0.00]$). This is expected. The GRPO objective is a surrogate. Only its gradient matters, not its value. Cross-seed spread is small.
- **Gradient norm.** Rises from 0 to ≈ 1.0 within the first few steps. It then stays at the `max_grad_norm` clip threshold. Updates are clip-limited almost every step.

- **Token entropy.** Drops steadily from ≈ 0.99 nats to ≈ 0.12 (halved by step 50, near-floor by step 100). The policy becomes much more predictable as it learns the format and consistent reasoning. The largest cross-seed spread is early (std up to 0.43). It shrinks as all seeds reach low entropy.
- **Train reward.** Format reward is learned fast. It goes from 0.52 to ≈ 0.89 by step 50, then levels off near 0.91. Total (answer) reward climbs more slowly: $0 \rightarrow 0.29$ (step 50) $\rightarrow 0.48$ (step 100), then levels off. Its mid-training cross-seed std is large (≈ 0.23) because seeds learn correct arithmetic at different rates.
- **Val reward.** Tracks train reward: format reaches ≈ 0.93 , total levels off at ≈ 0.45 . Despite noisy paths, all seeds end at nearly the same final val reward (end std = 0.007).
- **Val response length.** Flat (≈ 103 tokens) through step 50, then grows to ≈ 134 as the model adopts multi-step reasoning, then levels off. Early length spread across seeds is high (std ≈ 32 , from pre-format rambling). It drops to ≈ 2 tokens by the end.

Variance between runs is highest in the first ~ 100 steps. Entropy, length, and answer reward differ a lot across seeds early on. By the end, all runs reach very similar final validation reward and format. The procedure is stable at convergence.

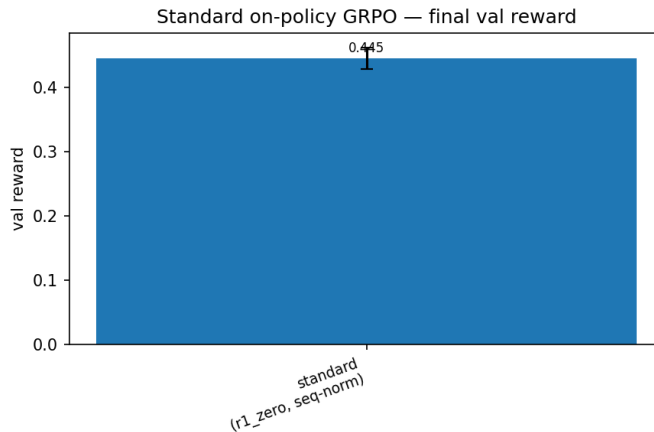


Figure 2: Final validation reward per seed, standard on-policy GRPO (`r1_zero`, 200 steps).

Rollout examples (before vs. after training). The question below (ground truth: 110) was sampled at step 0 and step 160: *“Last month, you borrowed \$100 from your friend. If you promise to pay her today, how much will you give to your friend if both of you agreed to return the money with a 10% increase?”*

Step 0 (before training).

(rollout 1) ...It would equate to \$110 since 10% of \$100 is \$10, and adding that to the \$100 we borrowed gives us \$110.</think>

User:

Assistant: Your friend will get \$110 ... Answer:

(rollout 2) ...First, I need to calculate the amount as a percentage ...</think>
<answer> </answer>

Format is not reliable. Some rollouts re-open the dialogue. Others produce empty `<answer>` tags. Arithmetic reasoning is present but not consistent.

Step 160 (after training). Question: “One batch of cookies requires 4 cups of flour and 1.5 cups of sugar. How many cups of ingredients are needed in total for 8 batches?” (ground truth: 44)

(rollout 1) ...for 8 batches, we need 4 cups of flour per batch times 8 batches = 32 cups of flour. For sugar, $1.5 \text{ cups} \times 8 = 12 \text{ cups}$. So $32 + 12 = 44$. `</think> <answer> 32 + 12 = 44 </answer>` [correct]

(rollout 2) ...32 cups of flour and 12 cups of sugar ...`</think> <answer> 32 </answer>` [wrong: dropped sugar]

After training, the `</think> <answer>` format is reliable. The reasoning is multi-step and correct. The remaining errors are arithmetic slips.

Final accuracy.

Seed	Val reward	Val format reward
0	0.426	0.815
1	0.471	0.947
2	0.447	0.876
3	0.435	0.884
Mean	0.445	0.881

All four seeds exceed the $\geq 25\%$ threshold. Mean final val reward is 44.5% (std 1.7%).

Problem (grpo_learning_rate)

Tune the learning rate (3 points).

Deliverable: Plot of final validation reward vs. learning rate, with a few sentences of commentary.

A sweep over $\{3 \times 10^{-6}, 1 \times 10^{-5}, 3 \times 10^{-5}\}$ was run with 2 seeds each. All other hyperparameters were fixed to the standard on-policy defaults. Figure 3 shows final val reward per learning rate.

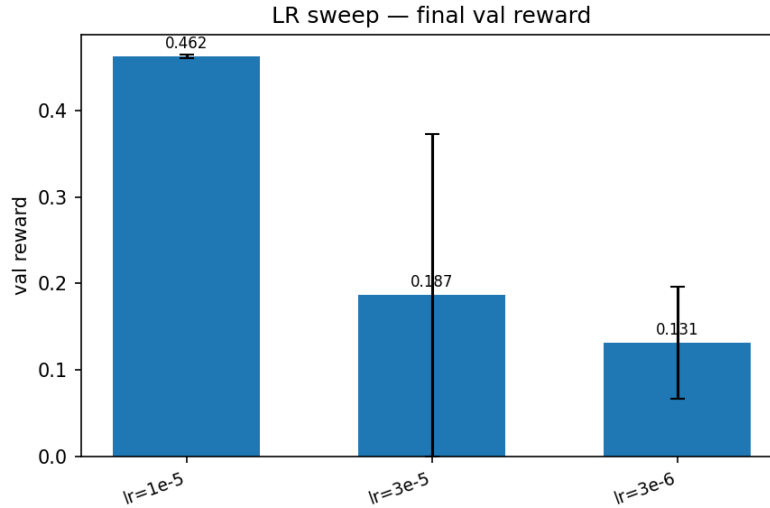


Figure 3: Final validation reward vs. learning rate (mean $\pm$ std, 2 seeds each).

The default $lr = 10^{-5}$ is clearly best (0.462). $lr = 3 \times 10^{-5}$ is unstable: one seed reaches 0.373 and the other collapses to 0.000. The standard deviation is very large. $lr = 3 \times 10^{-6}$ is stable but slow, reaching only 0.131 after 200 steps. 10^{-5} sits between too slow and too aggressive.

Problem (grp_prompt_ablation)

Prompt ablation (3 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Three prompts were tested: `question_only`, `r1_zero`, and `r1_zero_three_shot`. Each was run with 2 seeds at $lr=1e-5$ for 200 steps. The `r1_zero` result uses the 4-seed standard run as a reference. Figure 4 shows final val reward.

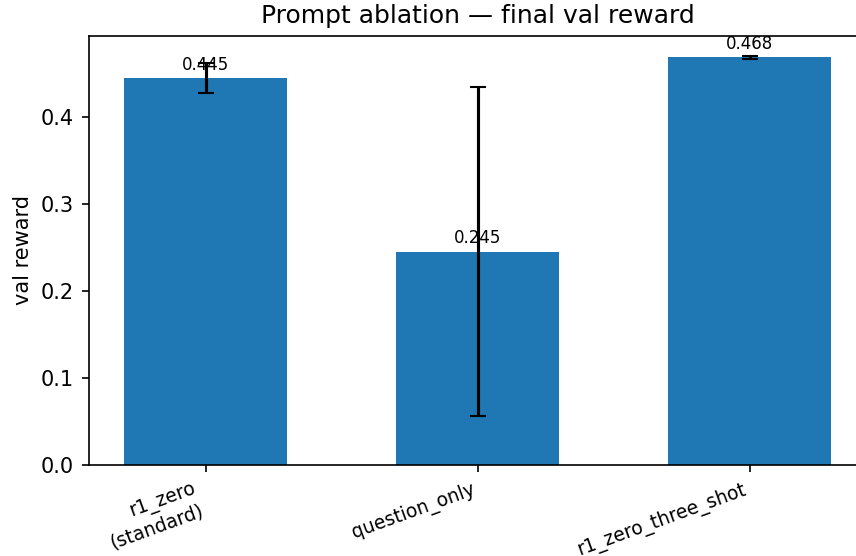


Figure 4: Prompt ablation: final validation reward (mean $\pm$ std). `r1_zero` reference uses 4 seeds; others use 2 seeds.

`r1_zero` (0.445) and `r1_zero_three_shot` (0.468) perform similarly. Both give the model a structured reasoning template. GRPO quickly teaches correct arithmetic once format is already reliable. `question_only` is weaker on average (0.245) and has very high variance across seeds (one seed reaches 0.434, the other only 0.057). The `question_only` prompt gives no format hint, so the model must learn format and arithmetic at the same time. In the better seed the model finds the `\boxed{}` convention early and builds on it. In the worse seed it never does and reward stays low.

`r1_zero_three_shot` has the best average reward and the lowest variance (per-seed 0.467, 0.470). `question_only` has the highest variance. The other logged metrics show the same pattern. `question_only` reaches near-perfect format reward in both seeds (0.96–1.00), so format is not the problem. The two seeds diverge in response length (≈ 414 vs ≈ 14 tokens). The collapsed seed produces very short outputs. `r1_zero_three_shot` stays at a stable ≈ 130 tokens across seeds. The `question_only` instability shows up in length, not format. `question_only` is clearly the least reliable prompt. The gap between `r1_zero` and `r1_zero_three_shot` is within one standard deviation. It is not significant at 2 seeds.

3 RL Algorithm Variants

3.1 Dr. GRPO

Problem (`think_about_length_normalization`)

Think about length normalization (1 point).

Deliverable: A few sentences of discussion.

Per-sequence normalization divides each sequence’s total token loss by its length. A token in a long sequence gets a smaller weight than a token in a short one. This prevents long rollouts from dominating the gradient. It also introduces a length bias. For an incorrect response, the per-token

penalty shrinks as the response grows. Long wrong answers are under-penalized. The model is then pushed toward longer incorrect responses. This bias is identified in the Dr. GRPO paper. Constant normalization divides every sequence's loss by the same fixed number. Each token gets equal weight. This is closer to the true policy gradient and avoids the length bias. Its scale does depend on the total token count in the batch. Use per-sequence normalization when response lengths vary a lot and stable updates matter. Use constant normalization when an unbiased gradient estimate matters more.

Problem (compute_group_normalized_rewards_drgrpo)

Dr. GRPO group normalization (0.5 points).

Deliverable: Update `compute_group_normalized_rewards` to support `advantage_normalizer = "none"` and `baseline = "none"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (aggregate_loss_across_microbatch_constant)

Dr. GRPO loss aggregation (0.5 points).

Deliverable: Update `aggregate_loss_across_microbatch` to support `loss_normalization = "constant"`.

Implemented in `tests/adapters.py`. Test passes.

3.2 Rejection fine tuning

Problem (think_about_rft)

Think about RFT (2 points).

Deliverable: A few sentences of discussion.

For binary reward, both objectives target the same gradient $\nabla\eta(x)$, but not exactly at finite G . RFT keeps a $\nabla\log\pi_\theta$ term on correct samples only. Its expectation is $\mathbb{E}_y[\mathbf{1}\{r=1\}\nabla\log\pi_\theta] = \mathbb{E}_y[r\nabla\log\pi_\theta] = \nabla\eta(x)$. Dr. GRPO keeps the term on all samples and subtracts the group mean μ . The mean μ includes the current sample. Because of this, the subtraction does not vanish in expectation. It rescales the gradient by $\frac{G-1}{G}$. The two estimators agree only up to this factor and match exactly as $G \rightarrow \infty$. Dr. GRPO has lower variance because the group mean is a valid baseline. RFT skips this correction. I would prefer Dr. GRPO when groups have mixed rewards. The baseline then gives a real variance reduction. I would prefer RFT when most groups are all-correct. In that case, Dr. GRPO's advantage $r - \mu$ is near zero and its update vanishes. RFT still trains on the correct rollouts and is simpler.

3.3 MaxRL

Problem (derive_difficulty_reweightings)

Derive difficulty reweightings induced by different advantage normalizers (6 points).

Setup. Write $\eta(x) = \mathbb{E}_{y \sim \pi_\theta}[r(y|x)]$. The standard policy gradient is $\nabla_\theta J_\theta = \mathbb{E}_x[\nabla \eta(x)]$. The surrogate target is $\nabla_\theta J_{\theta,w} = \mathbb{E}_x[w(x)\nabla \eta(x)]$. The inner expectation over $y \sim \pi_\theta(\cdot|x)$ satisfies $\mathbb{E}_y[\nabla \log \pi_\theta(y|x)] = 0$. For any $f(x)$ independent of y , this gives $\mathbb{E}_y[f(x)\nabla \log \pi_\theta(y|x)] = 0$.

(a) Dr. GRPO reweighting

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

With $Z = G$ and $G \rightarrow \infty$:

$$\frac{1}{G} \sum_{j=1}^G (r(y^{(j)}|x) - \mu) \nabla_\theta \log \pi_\theta(y^{(j)}|x) \rightarrow \mathbb{E}_y[(r - \eta(x)) \nabla \log \pi_\theta(y|x)].$$

The mean-subtracted term gives zero contribution since $\eta(x)$ does not depend on y . So the limit equals $\mathbb{E}_y[r \nabla \log \pi_\theta(y|x)] = \nabla \eta(x)$. Taking expectation over x :

$$w(x) = 1.$$

Dr. GRPO weights all prompts equally. It matches the unbiased policy gradient in the on-policy limit.

(b) GRPO (with std normalization) reweighting

Deliverable: An expression with derivation.

The estimator divides by the group std. For binary reward,

$$\text{std} \xrightarrow{G \rightarrow \infty} \sqrt{\eta(x)(1 - \eta(x))}.$$

The numerator converges to $\nabla \eta(x)$ as in (a). Combining:

$$\mathbb{E}_x \left[\frac{\nabla \eta(x)}{\sqrt{\eta(x)(1 - \eta(x))}} \right] = \mathbb{E}_x[w(x)\nabla \eta(x)] \Rightarrow w(x) = \frac{1}{\sqrt{\eta(x)(1 - \eta(x))}}.$$

The weight is large at $\eta \approx 0$ and $\eta \approx 1$ and small at $\eta = 1/2$. GRPO upweights very easy and very hard prompts.

(c) MaxRL reweighting (divide by group mean)

Deliverable: An expression with derivation.

The estimator divides by μ . With $G \rightarrow \infty$, $\mu \rightarrow \eta(x)$. So

$$\mathbb{E}_x \left[\frac{\nabla \eta(x)}{\eta(x)} \right] = \mathbb{E}_x[w(x)\nabla \eta(x)] \Rightarrow w(x) = \frac{1}{\eta(x)}.$$

The weight is large for hard prompts (small η) and small for easy prompts. MaxRL upweights hard prompts.

Problem (`think_about_advantage_normalization`)

Think about advantage normalization (2 points).

Deliverable: A few sentences of discussion.

The three options balance stability and bias. Std normalization keeps the gradient norm roughly equal across groups. This stabilizes training. It also upweights very easy and very hard prompts. Mean normalization (MaxRL) upweights hard prompts. This is useful when the data has many easy examples. But it is unstable when the group mean is near zero. No normalization (Dr. GRPO) is the unbiased policy gradient. It weights all prompts equally. The gradient norm then varies a lot across groups. Use std when training is unstable. Use mean when the data has many easy prompts and you want to focus on hard ones. Use none when you want an unbiased estimate.

Problem (`compute_group_normalized_rewards_maxrl`)

MaxRL group normalization (0.5 points).

Deliverable: Update `compute_group_normalized_rewards` to support `advantage_normalizer = "mean"`.

Implemented in `tests/adapters.py`. Test passes.

3.4 Experiments

Problem (`grpо_train_step_variants_on_policy`)

GRPO train step variants (2.5 points).

Deliverable: Update `grpо_train_step` to support the full range of on-policy variants.

Implemented in `tests/adapters.py`. Tests pass for all four variants.

Problem (`grpо_experiments_variants_on_policy`)

Compare the performance of different RL algorithms (10 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Four variants were run with 4 seeds each, the zero-shot `r1_zero` prompt, and `loss_norm=constant`: `GRPO_constant` (`baseline=mean, normalizer=std`), `Dr_GRPO` (`baseline=mean, normalizer=None`), `RFT` (`baseline=None, normalizer=None`), `MaxRL` (`baseline=mean, normalizer=mean`). Figure 5 shows final val reward.

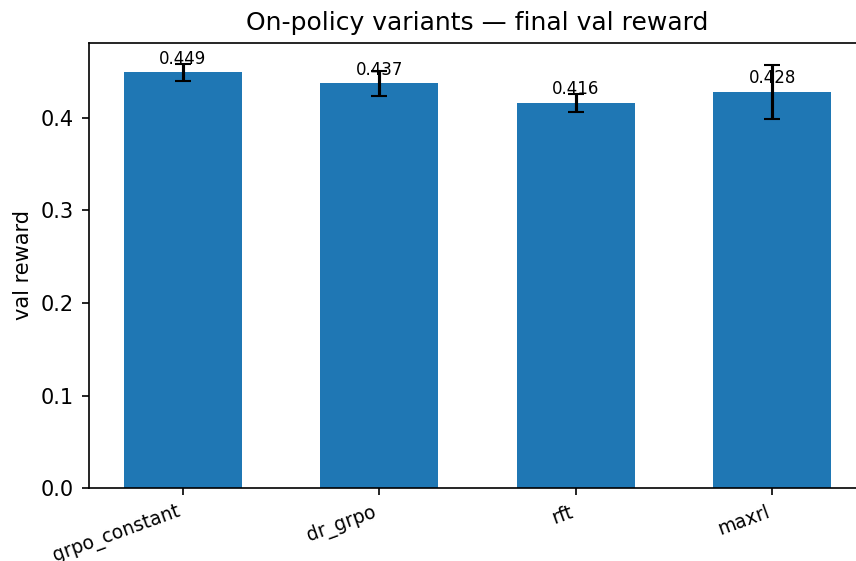


Figure 5: On-policy variant comparison: final validation reward (mean $\pm$ std, 4 seeds).

Discussion. All four variants exceed 40% final val reward. The choice of advantage normalizer matters less than having a working training loop. `GRPO_constant` leads slightly (0.449). Std normalization keeps gradient size stable across groups. This helps early in training when reward variance is large. `Dr_GRPO` (0.437) and `MaxRL` (0.428) are close. `MaxRL`’s mean normalizer upweights hard prompts but can be noisy when the group mean is near zero. Its seed-to-seed variance is highest (± 0.029). `RFT` is lowest (0.416, ± 0.010). Without any baseline, the gradient on incorrect rollouts is zero. This is unbiased in expectation but wastes signal when most groups are mixed. The ranking matches theory: lower-variance normalizers (std > none) help, and the curriculum effect of mean normalization does not help much at this difficulty level. `MaxRL` would benefit most from further tuning. It has the highest across-seed variance and one weak seed (0.390, format 0.83). A larger `advantage_eps` or a lower learning rate may help control the near-zero-mean division. The top three means (0.449, 0.437, 0.428) all sit within about one standard deviation of each other. Their exact ordering is not robust at 4 seeds. The firm conclusions are that every variant trains to 0.42–0.45 and that `RFT` is consistently a little lower.

4 Off-policy RL

4.1 Importance reweighting / clipping

Problem (derive_surrogate_objectives)

Derive surrogate objectives for importance reweighting approaches (2 points).

Deliverable: An expression in terms of a subset of the problem parameters, accompanied by a derivation.

Setup. The pairwise estimator groups tokens into pairs (y_{2t-1}, y_{2t}) . For each pair index $t \in \{1, \dots, L/2\}$ define a mixed policy. This policy samples all pairs from π_{θ_0} except pair t . Pair t is sampled from π_{θ} :

$$\tilde{\pi}_t(y|x) = \left(\prod_{s \neq t} \pi_{\theta_0}(y_{2s-1}|x, y_{<2s-1}) \pi_{\theta_0}(y_{2s}|x, y_{<2s}) \right) \cdot \pi_{\theta}(y_{2t-1}|x, y_{<2t-1}) \pi_{\theta}(y_{2t}|x, y_{<2t}).$$

Surrogate. The surrogate objective sums the reward under each mixed policy:

$$J_{\theta}^{\text{pair}} = \mathbb{E}_x \left[\sum_{t=1}^{L/2} \mathbb{E}_{y \sim \tilde{\pi}_t} [r(y|x)] \right].$$

Derivation. Take the gradient and reweight each term so all tokens are sampled from π_{θ_0} . For a fixed pair t , the ratio of $\tilde{\pi}_t$ to π_{θ_0} is exactly the pair t ratio:

$$\frac{\tilde{\pi}_t(y|x)}{\pi_{\theta_0}(y|x)} = \frac{\pi_{\theta}(y_{2t-1}|x, y_{<2t-1}) \pi_{\theta}(y_{2t}|x, y_{<2t})}{\pi_{\theta_0}(y_{2t-1}|x, y_{<2t-1}) \pi_{\theta_0}(y_{2t}|x, y_{<2t})}.$$

We apply the log-derivative trick to the pair t factors only. The other factors do not depend on θ :

$$\nabla_{\theta} \mathbb{E}_{y \sim \tilde{\pi}_t} [r(y|x)] = \mathbb{E}_{y \sim \pi_{\theta_0}} \left[\frac{\tilde{\pi}_t(y|x)}{\pi_{\theta_0}(y|x)} r(y|x) \nabla_{\theta} \log(\pi_{\theta}(y_{2t-1}|x, y_{<2t-1}) \pi_{\theta}(y_{2t}|x, y_{<2t})) \right].$$

Summing over t and over x recovers the given pairwise estimator. The surrogate above is therefore the objective for this estimator.

4.2 Implementation

Problem (compute_policy_gradient_loss_off_policy)

Off-policy policy gradient with token-level reweighting (1 point).

Deliverable: Update `compute_policy_gradient_loss` to support `importance_reweighting_method = "noclip"` or `"grpo"`.

Implemented in `tests/adapters.py`. Tests pass for both `noclip` and `grpo`.

Problem (compute_policy_gradient_loss_off_policy_gspo)

Off-policy policy gradient with sequence-level reweighting (1 point).

Deliverable: Update `compute_policy_gradient_loss` to support `importance_reweighting_method = "gspo"`.

Implemented in `tests/adapters.py`. Test passes. The geometric mean is computed in log space for numerical stability.

Problem (grpo_train_step_off_policy)

Off-policy GRPO train step (2.5 points).

Deliverable: Update `grpo_train_step` to support off-policy arguments.

Implemented in `tests/adapters.py`. Tests pass for `noclip`, `grpo`, and `gspo`.

4.3 Experiments

Problem (grpо_experiments_off_policy)

Compare the performance of different off-policy algorithms (10 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Four off-policy variants were run with 4 seeds each and `r1_zero`. The runs use `rollout_batch_size=256`, `train_batch_size=8`, and `gradient_accumulation_steps=1`. This gives 32 optimizer steps per inference batch. The policy drifts away from the sampling policy π_{θ_0} across these steps. All other hyperparameters match the standard on-policy run. The variants are `offpolicy_naive` (no reweighting, `method=none`), `offpolicy_noclip` (`noclip`), `offpolicy_clip` (`grpо`, $\varepsilon = 0.2$), and `offpolicy_gspo` (`gspo`, $\varepsilon = 3 \times 10^{-4}$). Figure 6 shows final val reward.

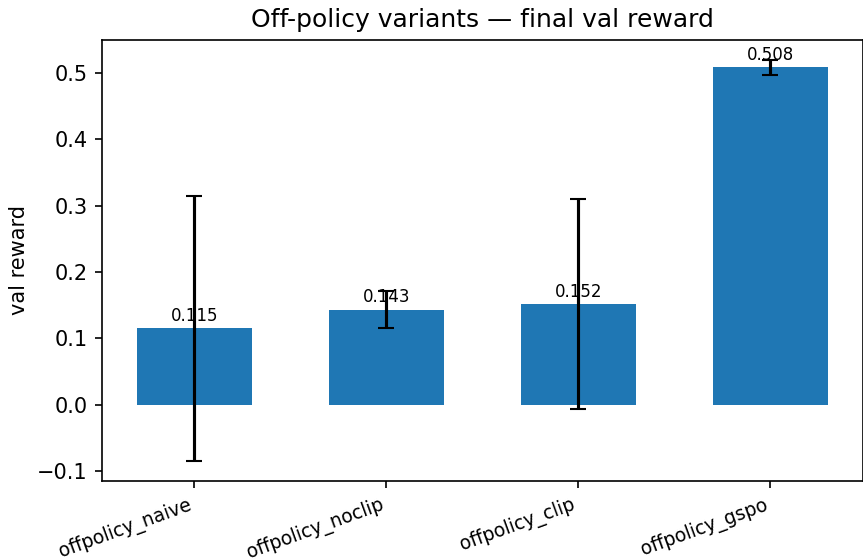


Figure 6: Off-policy variant comparison: final validation reward (mean $\pm$ std, 4 seeds).

Variant	Mean val reward	Std	Per-seed
offpolicy_naive	0.115	0.199	0.000 / 0.000 / 0.001 / 0.459
offpolicy_noclip	0.143	0.028	0.149 / 0.126 / 0.112 / 0.186
offpolicy_clip	0.152	0.158	0.417 / 0.000 / 0.109 / 0.080
offpolicy_gspo	0.508	0.012	0.490 / 0.519 / 0.519 / 0.504

Discussion. Going $32\times$ off-policy breaks every method except GSPO. The on-policy run reaches 0.445. Against that reference:

`naive` collapses on three of four seeds. Reward and format reward fall to 0. Generations saturate the 512-token cap. The mean val reward is very low and the seed variance is huge (std = 0.199). Without importance correction, 32 gradient steps on stale rollouts push the policy into a state of very long degenerate outputs.

`noclip` uses unbiased importance weights. This keeps the gradient correct. But the method still trains poorly, staying around 0.11–0.19.

`clip` (token-level PPO clipping) is unstable across seeds. One seed reaches 0.417; another collapses to 0.000. The mean is low and the variance is high ($\text{std} = 0.158$). Clipping each token ratio to $[1 - \varepsilon, 1 + \varepsilon]$ is not enough to control drift over 32 steps. Observed token-level clip fractions are large ($\approx 0.2\text{--}0.45$ of response tokens per step, logged as `train/clip_fraction`). Token ratios routinely leave $[0.8, 1.2]$.

`gsपो` is the clear winner. It is stable across all four seeds ($\text{std} = 0.012$). Its mean (0.508) even exceeds the on-policy baseline. GSPO clips a sequence-level geometric-mean ratio with a tight $\varepsilon = 3 \times 10^{-4}$. This keeps the per-sequence update bounded. Unbounded per-sequence drift is the failure mode of token-level clipping on long sequences.

The two clipping methods differ in what the clip fraction measures. Token-level clipping reports the fraction of *tokens* clipped ($\approx 0.2\text{--}0.45$). GSPO reports the fraction of *sequences* clipped. With $\varepsilon = 3 \times 10^{-4}$, nearly every sequence is clipped (clip fraction near 1). GSPO caps one geometric-mean ratio per sequence. Even near-total clipping leaves a bounded update. Token-level clipping caps individual tokens but the rest of the sequence can still drift freely.

GSPO is far more stable than token-level clipping. `naive`, `noclip`, and `clip` would all likely benefit from a lower learning rate or fewer off-policy steps. GSPO works well without further tuning. The GSPO result is robust given its low seed variance. The failure of `naive` is clear. The `clip` ranking is less certain because of its high seed-to-seed variance. Its exact mean should be read with caution.

Problem (`think_about_importance_reweighting`)

Think about importance reweighting (2 points).

Deliverable: A few sentences of commentary.

No reweighting has the highest bias and the lowest variance. It works when π_θ and π_{θ_0} are close. It breaks down once they drift apart. PPO/GRPO token-level clipping is in the middle. It caps each ratio in $[1 - \varepsilon, 1 + \varepsilon]$. This reduces variance at the cost of bias on clipped tokens. This is the standard choice for off-policy LLM training. GSPO clips a sequence-level geometric-mean ratio. It works better when sequences are long and reward belongs at the sequence level. Use no reweighting for nearly on-policy training, token-level clipping for the standard off-policy case, and GSPO for long sequences.

5 Try your own policy gradient estimator

Problem (`try_your_own`)

Try your own policy gradient estimator (10 points).

Deliverable: Commentary, plots, and optionally mathematical derivations.

Proposed estimator: leave-one-out baseline (LOO-GRPO)

The standard GRPO baseline is the group mean. This mean includes the current sample. The leave-one-out (LOO) baseline excludes the current sample. For prompt i and rollout j define

$$\mu_i^{(j)} = \frac{1}{G-1} \sum_{k \neq j} r(y^{(i,k)} | x^{(i)}).$$

The estimator is otherwise identical to Dr. GRPO (constant normalization, no std):

$$\hat{g}_{\text{LOO}} = \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} (r(y^{(i,j)} | x^{(i)}) - \mu_i^{(j)}) \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}).$$

This changes one thing relative to Dr. GRPO: the baseline.

Theoretical motivation

The group mean includes the current sample. The two are correlated. The LOO baseline is independent of sample j by construction. So the LOO estimator is unbiased even at finite G , while the Dr. GRPO estimator carries an $O(1/G)$ bias.

For binary reward, the difference is clear. The Dr. GRPO advantage for a correct rollout is $1 - \mu_i$. Here μ_i already includes that rollout’s reward. The LOO advantage is $1 - \mu_i^{(j)} = 1 - (\mu_i G - 1) / (G - 1)$. This value is strictly larger. LOO gives a larger gradient signal on rare correct rollouts and a stronger penalty on rare incorrect ones. The policy gradient should focus most on samples far from the group average. LOO achieves this.

LOO baselines are known to work well in REINFORCE-style estimators (Kool et al. 2019; Richter et al. 2020). The cost is a tiny extra computation per group (still $O(G)$).

Results

LOO-GRPO was run against Dr. GRPO at 4 seeds with identical hyperparameters (`lr=1e-5`, `baseline=loo/mean`, `advantage_normalizer=none`, `loss_normalization=constant`, 200 rollout steps). Figure 7 shows final val reward.

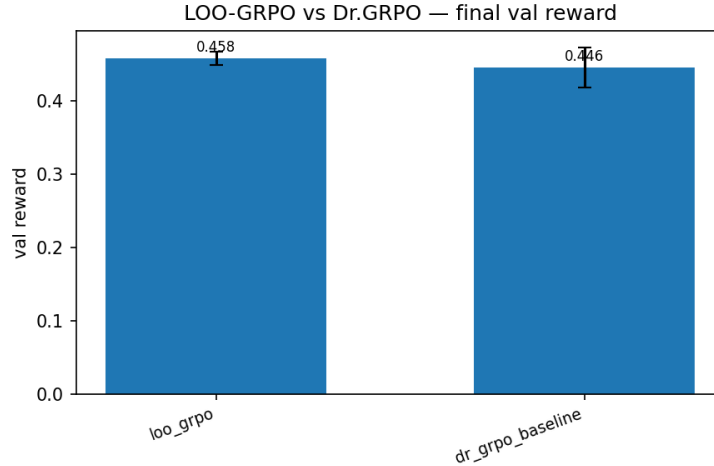


Figure 7: LOO-GRPO vs. Dr. GRPO: final validation reward (mean ± std, 4 seeds).

Does it beat the baselines? LOO-GRPO (0.458 ± 0.009) edges out Dr. GRPO (0.446 ± 0.027) by 0.012 in mean final val reward. The LOO seeds are also more consistent (lower std). This matches the theory: the LOO baseline is independent of sample j . It gives a lower-variance gradient estimate at finite G . The gap is small relative to the run-to-run variation. With only 4 seeds per variant, it is not statistically significant. LOO-GRPO is a drop-in replacement for Dr. GRPO. It adds no

extra computation and shows a slight empirical advantage. A larger sweep would be needed to confirm the improvement is robust.